

hackathon-github-top20\10-f3bb2993d9ff49df-agent-rebuttal.log

对应微信名：FEVER

申诉复核：10-f3bb2993d9ff49df-agent

原反作弊结果：result/hackathon-github-top20/10-f3bb2993d9ff49df-agent.log

申诉文件：rebuttal/fever/rebuttal.md

作答路径：workspace/hackathon-github-top20/10-f3bb2993d9ff49df-agent

复核范围：仅核实申诉所述代码内容是否存在，以及申诉是否回应原反作弊结果；不重新裁定是否作弊。

一、结论摘要

1. 申诉关于“提交中存在正式测试目录探测、helpers.ts / *.spec.ts 读取和静态解析逻辑”的核心陈述属实。
2. 申诉对函数名的表述不准确：实际函数名是 `_find_tests_dir()`，不是 `tests_dir()`；`tests_dir` 是调用阶段使用的局部变量。
3. 申诉所述 p19/p20、90.6/40.6、复制 helpers.ts 等文字确实存在于源码注释或预置规则中，但当前材料没有对应的平台运行日志，不能据此独立确认历史事件和分数。
4. 当前提交目录没有 .git，且未找到提交 `1c408b`，无法核实申诉所称 2026-09-22 git 合规审查历史。
5. 当前提交及本复核材料中未找到 `check_template_compliance.py`，申诉关于该脚本存在的陈述不能由当前材料支持。
6. 累计提交 22 次、32 项测试通过数、58 个功能项仅 2 项通过等陈述没有随附平台日志，无法核实。
7. 原反作弊日志中的两项 ERROR 和一项 WARNING 均未在申诉中得到实质回应。申诉集中讨论的是“测试读取逻辑”，而原日志的实际依据是：
 - ERROR-001：预置并默认注入登录页“Sign in”冲突的具体实现提示；
 - ERROR-002：HTML 报告从外部站点加载字体 CSS 和 ECharts JavaScript；
 - WARNING-001：独立视觉模型、密钥及网关配置。

二、申诉所述测试读取代码核验

(一) 探测 `/workspace/tests`：存在

位置：verify/verify.py:672-716

关键源码：

```
1 def _find_tests_dir(requirements_dir: Path, output_dir: Path) -> Path |  
  None:  
2     """Locate the official test specs (staging/tests-context) if visible.  
3     ...  
4     """  
5     candidates: list[Path] = []  
6     export = os.environ.get("ARC_EXPORT_DIR", "")  
7     if export:  
8         candidates.append(Path(export) / "staging" / "tests-context")
```

```

9
10     # The platform runner mounts the graded suite at /workspace/tests. The
agent's
11     # own shell can read it (p20 copied helpers.ts verbatim into e2e-tmp/)
but the
12     # gate could not, so every test-derived contract was inert on the
platform
13     # while parsing fine in the sim – observed as `test_flow_source: null`.
14     for env_key in ("ARC_TESTS_DIR", "ARCBENCH_TESTS_DIR"):
15         value = os.environ.get(env_key, "")
16         if value:
17             candidates.append(Path(value))
18     candidates.append(Path("/workspace/tests"))
19     ...
20     if (base / "helpers.ts").is_file() and any(base.glob("*.spec.ts")):
21         return base

```

该代码明确将 `/workspace/tests`、环境变量指定目录、`staging/tests-context` 和邻近 `tests` 目录加入候选，并以同时存在 `helpers.ts` 与 `*.spec.ts` 作为识别条件。

(二) 读取并解析正式测试文件：存在

位置：verify/verify.py:719-777

关键源码：

```

1 def _test_flow_clauses(tests_dir: Path) -> list[dict[str, Any]]:
2     """Parse official test specs into seed initial-state expectations (O-
B)."""
3     clauses: list[dict[str, Any]] = []
4     fixtures_block = _fixtures_block(tests_dir)
5     leaf = _fixture_leaf_map(tests_dir)
6     specs = sorted(tests_dir.glob("*.spec.ts"))
7     ...
8     for spec in specs:
9         try:
10             text = spec.read_text(encoding="utf-8", errors="ignore")
11         except OSError:
12             continue
13     ...

```

位置：verify/verify.py:847-932

关键源码：

```

1 def _official_query_shapes(tests_dir: Path | None) -> dict[str, Any]:
2     """How the official suite resolves accessible names (OPT-2 evidence
layer)."""
3     ...
4     helpers = tests_dir / "helpers.ts" if tests_dir else None
5     if not helpers or not helpers.is_file():
6         return out
7     try:
8         htext = helpers.read_text(encoding="utf-8", errors="ignore")
9     except OSError:
10         return out
11     ...

```

```

12     for path in sorted(set(list(tests_dir.glob("*.ts")))):
13         try:
14             text = path.read_text(encoding="utf-8", errors="ignore")
15         except OSError:
16             continue

```

上述逻辑会读取 `helpers.ts` 和测试目录下的 TypeScript 文件，解析测试操作序列、fixture 和 locator/query 形式。

(三) 解析结果进入验证报告：存在

位置：verify/verify.py:2217-2235

关键源码：

```

1  # O-B: official-test flow initial-state contract (visible tests-context
2  # only; static parse, never executes the specs).
3  tests_dir = _find_tests_dir(requirements_dir, output_dir)
4  report["level2"]["test_flow_source"] = str(tests_dir) if tests_dir else None
5  flow_clauses = _test_flow_clauses(tests_dir) if tests_dir else []
6  report["level2"]["test_flow_clauses"] = flow_clauses
7  ...
8  query_shapes = _official_query_shapes(tests_dir)
9  report["level2"]["query_shapes"] = {
10     "source": query_shapes.get("source"),
11     "landmark_button_count": len(query_shapes.get("landmark_button") or []),
12     "global_count": len(query_shapes.get("global") or []),
13     ...
14 }

```

因此，申诉关于测试目录探测和静态解析代码存在的核心陈述成立。代码注释同时明确表示“static parse, never executes the specs”，即这里核实的是读取和解析，不是执行测试。

三、原反作弊 ERROR-001 的具体实现及申诉遗漏情况

(一) 预置了什么

预置的不是一套已经完成的登录页面源码，而是一条可直接指导模型实现登录页的具体规则：当 `/login` 页面的页头入口和表单提交按钮都叫“Sign in”时，将页头入口做成普通链接，使表单中的“SIGN IN”成为唯一具有 button 角色的同名控件。

位置：main.py:448-547，具体为 main.py:500-520

关键源码：

```

1  7. Control roles match official semantics: navigation/list items/primary
2  CTAs a test may click are <button> (or explicit role="button"), not
3  bare links;
4  ...
5  EXCEPTION (SKILL rule E.5, narrowed after p20): a chrome entry may lose
6  its button role ONLY when the graded suite resolves that name globally
7  AND the same name is a page action on the page – e.g. a header "Sign in"
8  as a plain link on /login so the login form's "SIGN IN" submit is the
9  only button with that name (observed p13 prestashop: 12 account/checkout
10 tests failed without it).

```

对应实现效果可概括为：

```
1 <!-- 页头入口只作为链接 -->
2 <a href="/login">Sign in</a>
3
4 <!-- 登录表单提交控件保留按钮角色 -->
5 <button type="submit">SIGN IN</button>
```

这样按 button 角色和“Sign in”名称查找控件的测试只会命中表单提交按钮。预置文字还明确说明该规则来自以往 PrestaShop 账户/结账测试失败经验，而不是仅从本题需求文本自然复述而来。

本题题面确实包含对应需求。位置：example/hackathon/github/requirements.yaml:146-152

题面内容：

```
1 - id: REQ-1-1-2
2   name: Sign In with an Existing Account
3   ...
4   description: |-
5     ... The sign-in page is the form on the account-access page containing
6     "Username or email", "Password", and "Sign in".
```

(二) 注入到了哪里

该规则属于 `_FULL_CONTRACT_ITEMS`，默认执行档位为 L0：

位置：main.py:44

```
1 ARCBENCH_EFFORT_DEFAULT = 0
```

L0 的 `_contract_section()` 返回完整预置契约：

位置：main.py:613-633

```
1 def _contract_section(effort: int) -> str:
2     ...
3     if effort == 0:
4         return (
5             _CONTRACT_HEADER
6             + _FULL_CONTRACT_ITEMS
7             + ...
8         )
```

每一个模块的 prompt 都会追加该契约：

位置：main.py:650-676

```
1 def _module_prompt(...):
2     ...
3     prompt += "\n\n" + _contract_section(effort)
4     if visual_block:
5         prompt += "\n" + visual_block
6     return prompt
```

随后，该 prompt 被交给负责生成应用代码的 Codex 线程：

位置: main.py:873-888

```
1 thread = codex.thread_start(  
2     cwd=str(output_dir),  
3     sandbox=Sandbox.full_access,  
4     approval_mode=ApprovalMode.deny_all,  
5     model=model,  
6     developer_instructions=_developer_instructions(...),  
7 )  
8 prompt = _module_prompt(module, requirements_dir, completed_ids, skills_dir,  
9     visual_block)  
10 ...  
11 result = _run_turn(thread, prompt, ...)
```

位置: main.py:823-845

```
1 if streaming_available(thread):  
2     handle = thread.turn(prompt)  
3     ...  
4 result = thread.run(prompt)
```

完整链路为:

```
1 ` _FULL_CONTRACT_ITEMS`  
2 -> `_contract_section(effort)`  
3 -> `_module_prompt(...)`  
4 -> `_run_turn(thread, prompt, ...)`  
5 -> `thread.turn(prompt)` / `thread.run(prompt)`  
6 -> Codex 在 output_dir 中生成或修改参赛应用代码。
```

(三) 申诉是否回应了这一项

没有。申诉没有提及 `/login`、页头“Sign in”、表单“SIGN IN”、按钮角色冲突、`_FULL_CONTRACT_ITEMS`、`_contract_section()` 或默认 prompt 注入链路。

申诉提到曾做“模板合规自查”和废弃“按需求预生成 seed/组件/页面骨架”，但这是另一项概括性历史陈述，并未回应原日志指出的上述具体登录实现提示。

四、原反作弊 ERROR-002 的外部资源加载方式、用途及申诉遗漏情况

(一) 外部字体加载

位置: report/index.html:8-9

关键源码:

```
1 <link rel="preconnect" href="https://miaoda.feishu.cn">  
2 <link href="https://miaoda.feishu.cn/fonts/css?  
   family=Noto+Sans+SC:wght@300;400;500;700;900&display=swap" rel="stylesheet">
```

浏览器打开该 HTML 时:

1. `preconnect` 提前与 `miaoda.feishu.cn` 建立网络连接;

2. `rel="stylesheet"` 请求并加载该站点返回的字体 CSS;
3. 字体 CSS 还可能继续请求其声明的实际字体文件。

字体的使用位置: report/index.html:18

```
1 body {  
2   font-family: 'Noto Sans SC', system-ui, -apple-system, 'Segoe UI',  
3               'Microsoft YaHei', sans-serif;  
4   ...  
5 }
```

目的: 让中文项目说明报告优先使用 Noto Sans SC, 获得统一的中文排版效果。网络资源不可用时, 浏览器会回退到系统字体, 报告主体仍可阅读。

(二) 外部 ECharts JavaScript 加载

位置: report/index.html:437

关键源码:

```
1 <script src="https://cdn.jsdelivr.net/npm/echarts@5.5.1/dist/echarts.min.js">  
  </script>
```

浏览器打开报告时会连接 `cdn.jsdelivr.net`, 下载 ECharts 5.5.1 JavaScript, 并在页面中执行该第三方代码。

ECharts 的实际使用位置: report/index.html:439-460

```
1 (function(){  
2   var el = document.getElementById('sizeChart');  
3   if(!el || typeof echarts === 'undefined') return;  
4   var chart = echarts.init(el);  
5   var cats = [  
6     {name:'skills 技能包 (12 文件)', v:55.7, c:'#0e9f8a'},  
7     {name:'arcbench-agent-runtime SDK (9 文件)', v:46.8, c:'#2563eb'},  
8     {name:'template 起步应用源码 (25 文件)', v:22.5, c:'#d97706'},  
9     {name:'入口 main.py 等 (3 文件)', v:8.9, c:'#7a8ba1'},  
10    {name:'examples 示例 (2 文件)', v:4.7, c:'#2e9e5b'}  
11  ];  
12  chart.setOption({  
13    ...  
14    series:[{  
15      type:'bar',  
16      ...  
17    }]  
18  });  
19  window.addEventListener('resize', function(){ chart.resize(); });  
20 })();
```

图表容器位置: report/index.html:241-244

```
1 <h3><span class="h3tag">■ </span>组件体量对比（源码，不含依赖锁文件）</h3>
2 <div class="diag">
3   <div id="sizeChart" class="chart"></div>
4   <div class="cap">各组件源码体积（KB）与文件数...</div>
5 </div>
```

目的：在随附项目说明报告中绘制“组件体量对比”横向柱状图。如果 ECharts 下载失败，代码因 `typeof echarts === 'undefined'` 直接返回，图表不显示，但报告的其他静态内容仍可显示。

这些请求来自仓库内随附的 `report/index.html`，不是生成出来的 GitHub 业务应用本身；但只要浏览器加载该报告，就会按 HTML 标签尝试连接外部站点并下载资源。

（三）申诉是否回应了这一项

没有。申诉没有提及：

- `miaoda.feishu.cn`；
- `cdn.jsdelivr.net`；
- Noto Sans SC 外部字体；
- ECharts 外部 JavaScript；
- `report/index.html` 的外部资源加载行为。

五、最终复核意见

申诉所述“测试读取逻辑存在”与仓库代码相符，但该逻辑不是原反作弊日志两项 ERROR 和一项 WARNING 的判定对象。申诉没有针对原日志实际列出的登录实现提示注入、HTML 外部资源加载和独立视觉模型配置作出解释或反驳。

因此，“反作弊结果的具体内容在申诉中没有提及”这一观察属实。该结论仅说明申诉与原检测依据不对应，不在本复核中重新判断各项行为是否构成作弊。

hackathon-github-top20\06-38c8627442a440eb-agent-rebuttal

对应微信名：e0_7

申诉复核：06-38c8627442a440eb-agent

原反作弊结果：result/hackathon-github-top20/06-38c8627442a440eb-agent.log

申诉文件：rebuttal/qfzy/rebuttal.md

作答路径：workspace/hackathon-github-top20/06-38c8627442a440eb-agent

复核范围：说明原结果 ERROR-001 和 ERROR-002 对应的具体代码行为、预置内容及进入生成流程的位置。按要求不复核 ERROR-003，也不重新裁定是否作弊。

一、结论摘要

ERROR-001 和 ERROR-002 是两个不同层次的预置机制：

1. ERROR-001 是“预置可运行源码”：仓库的 `scaffold/` 已经包含登录表单、凭据匹配、登录状态保存、当前用户解析、账户区域切换和退出动作。默认生成流程会先把这些文件复制到临时应用 `stage/`，再让模型修改；模型未覆盖的 `scaffold` 文件会直接保留并发布。
2. ERROR-002 是“预置模型指令和源码上下文”：`dialect.md` 提前规定登录/退出控件角色、种子账户密码、公共/私有资源可见性、会话存储方式等实现策略。生成请求会无条件把 `dialect.md` 放

在题面要求之前，并将 scaffold 全部源码作为 JSON 附加到同一条模型 user message 中，要求模型“extend it, never replace it”。

- ERROR-001 的 scaffold 并未完整实现 GitHub 题目的全部 Identity and Access 要求，但已经提供一套可以实际运行的通用认证与浏览器会话骨架。
- 申诉内容只请求提供具体审核理由，没有针对以下代码行为作出实质说明。

二、ERROR-001：预置的可运行认证与会话代码

（一）预置登录页面

位置：scaffold/frontend/public/app.js:30-35

源代码：

```
1 login() {
2   return `

# 


```

该页面不是占位注释，而是包含可提交表单、邮箱输入框、密码输入框和登录按钮的实际 UI。`data-action="login"` 会进入 scaffold 的事件分派机制。

（二）预置凭据匹配和成功/失败登录分支

位置：scaffold/frontend/public/app.js:41-47

源代码：

```
1 actions: {
2   login({ form }) {
3     const u = ARC.state.users.find(
4       (x) => x.email === form.email && x.password === form.password
5     );
6     if (!u) {
7       ARC.notify('Invalid email or password');
8       return;
9     }
10    ARC.state.session = u.id;
11    ARC.save();
12    ARC.navigate('#/');
13  },
14  logout() {
15    ARC.state.session = null;
16    ARC.save();
17    ARC.navigate('#/');
18  },
19 }
```

具体行为：

- 在 `ARC.state.users` 中匹配邮箱和密码；
- 匹配失败时显示错误消息；
- 匹配成功时把用户 ID 写入 `ARC.state.session`；

4. 调用 `ARC.save()` 保存状态;
5. 登录或退出后导航回首页;
6. 退出时将 session 清空。

(三) 预置了可直接登录的种子账户

位置: `scaffold/frontend/public/seed.js:3-6`

源代码:

```
1 window.SEED = {
2   session: null,
3   users: [
4     { id: 'u1', name: 'Demo User', email: 'demo@example.com', password:
      'password' }
5   ],
6   items: [{ id: 'i1', title: 'First item', body: 'Seeded row' }]
7 };
```

因此原始 scaffold 本身就可以使用 `demo@example.com` 和 `password` 通过前述登录逻辑登录。

(四) 预置登录前后的账户区域和退出按钮

位置: `scaffold/frontend/public/app.js:5-13`

源代码:

```
1 window.LAYOUT = {
2   brand: 'App',
3   nav: [{ label: 'Home', hash: '#/' }, { label: 'Items', hash: '#/items' }],
4   sidebar: [{ label: 'Items', hash: '#/items' }],
5   footer: 'Footer',
6   account: (user) => user
7     ? `${ARC.esc(user.name)}</span>
8       <button type="button" data-action="logout">Sign Out</button>`
9     : `
```

行为是:

- 未登录时显示 `Log in` 链接;
- 已登录时显示当前用户名和 `Sign out` 按钮;
- `Sign out` 触发前述 `logout()` 动作。

(五) 预置跨刷新和新标签页保留登录状态

位置: `scaffold/frontend/public/lib/arc.js:7-17`

源代码注释及状态拆分:

```

1 // Store: records are shared through the server; the sign-in and view state
2 // stay in this browser.
3 // `session` (the signed-in user's id; localStorage, so one sign-in per
4 // browser session that survives reloads and new tabs) ...
5
6 const SEEDED = new Set(Object.keys(window.SEED || {}));
7 const isShared = (k, v) =>
8   k !== 'session' && k !== 'ui' && (SEEDED.has(k) || Array.isArray(v));

```

读取本地会话，位置：scaffold/frontend/public/lib/arc.js:32-39

```

1 function readLocal() {
2   let local = {};
3   try {
4     local = JSON.parse(sessionStorage.getItem('arc:local:v2') || '{}') ||
5     {};
6   } catch (e) {}
7   try {
8     const s = localStorage.getItem('arc:session:v2');
9     if (s !== null) local.session = JSON.parse(s);
10  } catch (e) {}
11  return local;
12 }

```

保存本地会话，位置：scaffold/frontend/public/lib/arc.js:51-57

```

1 ARC.save = function () {
2   const { shared, local } = split(ARC.state);
3   const { session, ...view } = local;
4   try {
5     sessionStorage.setItem('arc:local:v2', JSON.stringify(view));
6   } catch (e) {}
7   try {
8     localStorage.setItem(
9       'arc:session:v2',
10      JSON.stringify(session === undefined ? null : session)
11    );
12   } catch (e) {}
13   sync('PUT', JSON.stringify(shared));
14 };

```

登录用户 ID 存入 `localStorage`，因此同一浏览器刷新页面或打开新标签页后仍可读取。其他视图状态写入 `sessionStorage`。

(六) 预置当前用户解析

位置：scaffold/frontend/public/lib/arc.js:111-115

源代码：

```

1 ARC.currentUser = function () {
2   const s = ARC.state;
3   if (!s || !s.session) return null;
4   return (s.users || []).find((u) => u.id === s.session) || null;
5 };

```

共享布局在每次渲染时调用 `ARC.currentUser()`，据此决定显示登录链接还是用户名和退出按钮。

(七) 预置跨浏览器共享业务记录的服务端状态接口

位置: scaffold/backend/server.js:19-36

源代码:

```
1 // The shared records every browser session reads (GET /api/state) and
2 // writes (PUT /api/state). A fresh process starts from the frontend's
3 // seed.js; `session` and `ui` belong to each browser, never here.
4
5 function loadSeed() {
6   ...
7   const shared = JSON.parse(JSON.stringify(seed));
8   delete shared.session;
9   delete shared.ui;
10  return shared;
11 }
12 let STATE = loadSeed();
```

位置: scaffold/backend/server.js:52-61

```
1 if (url.pathname === '/api/state') {
2   if (req.method === 'GET') return json(res, 200, STATE || {});
3   if (req.method === 'PUT') {
4     const body = await readBody(req);
5     if (!body || typeof body !== 'object' || Array.isArray(body))
6       return json(res, 400, { error: 'state must be an object' });
7     STATE = body;
8     return json(res, 200, { ok: true });
9   }
10 }
```

业务记录由 `/api/state` 在浏览器之间共享，登录 session 和 UI 状态仍归各浏览器本地所有。

三、ERROR-001：上述源码进入最终应用的位置和链路

(一) scaffold 默认启用

位置: main.py:84-89

源代码:

```
1 SCAFFOLD = os.environ.get("ONESHOT_SCAFFOLD", "1") != "0"
2 SCAFFOLD_DIR = HERE / "scaffold"
3 SCAFFOLD_KEEP = {
4   "frontend/public/lib/arc.js": (
5     "ARC.layout", "ARC.openDialog", "ARC.openPopover", "ARC.notify",
6     "ARC.rowActions", "ARC.render", "ARC.onPaste",
7     "ARC.pasteFromClipboard"
8   ),
9   "frontend/public/app.js": ("window.LAYOUT", "window.APP"),
10  "frontend/public/seed.js": ("window.SEED",)
11 }
```

ONESHOT_SCAFFOLD 缺省值是 1，所以没有显式关闭时会使用仓库内的 scaffold。

(二) 读取 scaffold 全部非隐藏文件

位置: main.py:1194-1222

源代码:

```
1 def read_template(root):
2     ...
3     blobs, total = {}, 0
4     for directory, dirs, names in os.walk(root, followlinks=False):
5         ...
6         data = path.read_bytes()
7         ...
8         blobs[path.relative_to(root).as_posix()] = data
9         if not {"frontend/package.json", "backend/package.json"} <=
blobs.keys():
10             raise ValueError(...)
11     return blobs
```

(三) 在模型生成前写入临时应用 stage

位置: main.py:1473-1492

源代码:

```
1 snapshot = read_template(template) if template is not None else {}
2 ...
3 scaffold = (
4     read_template(SCAFFOLD_DIR)
5     if template is None and SCAFFOLD and SCAFFOLD_DIR.is_dir()
6     else {}
7 )
8 messages = draft_messages(
9     requirements,
10    scaffold,
11    snapshot if template is not None else None,
12    prerequisites
13 )
14 ...
15 stage = Path(temp) / "app"
16 stage.mkdir()
17 ...
18 for name, data in (**scaffold, **snapshot).items():
19     target = resolve_path(stage, name)
20     target.parent.mkdir(parents=True, exist_ok=True)
21     target.write_bytes(data)
```

因此，登录、退出和会话文件在模型首次输出之前已经存在于待发布应用目录中。

(四) 模型只覆盖它实际输出的文件

位置: main.py:1411-1419

源代码:

```
1 for item in batch:
2     target = resolve_path(stage, item["path"])
3     item["path"] = target.relative_to(stage.resolve()).as_posix()
4     if target in emitted:
5         files[:] = [f for f in files if f["path"] != item["path"]]
6     emitted.add(target)
7     write_one(stage, item)
8     files.append(item)
```

模型没有输出的 scaffold 文件不会被删除, 会继续留在 stage 中。

(五) 检查核心 scaffold 结构必须保留

位置: main.py:1281-1299

源代码:

```
1 def scaffold_gaps(stage):
2     ...
3     for ref in ("seed.js", "lib/arc.js", "app.js"):
4         if ref not in html:
5             gaps.append(...)
6     for name, symbols in SCAFFOLD_KEEP.items():
7         path = stage / name
8         text = path.read_text(...) if path.is_file() else ""
9         missing = [sym for sym in symbols if sym not in text]
10        if not path.is_file():
11            gaps.append(f"{name} is missing; the scaffold requires it")
12        elif missing:
13            gaps.append(
14                f"{name} no longer defines {'', '.join(missing)}"; "
15                "the scaffold's primitives must stay defined"
16            )
```

位置: main.py:2128-2134

```
1 def static_gaps(stage, requirements, scaffold):
2     ...
3     if scaffold:
4         gates["scaffold"] = scaffold_gaps(stage)
```

(六) 发布 stage

位置: main.py:1501-1529

源代码:

```

1 conversation = draft_application(
2     stage, files, messages, model, base, key, budget, template
3 )
4 ...
5 repair_delivery(
6     stage, files, conversation, model, base, key, budget, template,
7     requirements, scaffold=bool(scaffold), tests=tests
8 )
9 ...
10 publish(stage, out, template, snapshot, stamp)

```

完整链路：

```

1 仓库 scaffold/
2 -> `read_template(SCAFFOLD_DIR)`
3 -> 写入临时 `stage/`
4 -> 模型增加或覆盖部分文件
5 -> scaffold 保留检查及修复
6 -> `publish(stage, out, ...)`
7 -> 最终应用。

```

四、ERROR-001 与题面要求的对应程度

本题 `example/hackathon/github/requirements.yaml` 明确包含：

- Identity and Access;
- Sign In with an Existing Account;
- 登录成功后创建并保持浏览器会话;
- Sign Out and End the Current Web Session;
- 登录状态影响保护资源和公共/私有资源可见性。

scaffold 已经实现的部分包括：

- 登录表单;
- 邮箱、密码匹配;
- 登录成功与失败分支;
- 当前用户状态;
- 登录前后账户区域变化;
- 退出;
- 刷新和同一浏览器新标签页后的会话恢复;
- 浏览器之间共享业务数据。

scaffold 尚未完整实现的题目细节包括：

- 用户名或邮箱均可登录;
- 题面要求的统一错误消息 `Invalid credentials`;
- 服务端唯一 session ID 和 active 状态;
- `Account menu -> Sign out -> Confirm sign out/Cancel` 对话框流程;
- 完整的保护页面权限判断;
- 密码更新、恢复等其他 Identity and Access 功能。

因此，ERROR-001 所称“预置登录、退出和浏览器会话实现”是指一套可运行的通用认证骨架，而不是已经完成了本题全部认证功能。

五、ERROR-002：预先编写的认证和可见性策略提示

(一) 预置登录、退出等控件的角色规则

位置: dialect.md:13-20

原文:

```
1  ## 1a. When the requirement names no role: buttons for actions,  
2  links only for navigation  
3  
4  A role the requirement names is the contract. Where it names none,  
5  helpers assert session and action controls by ROLE  
6  (`getByRole('button', {name: /sign out/i})` right after login).  
7  One large-task run rendered its sign-out control as `` and lost  
8  73 of 135 tests, because every logged-in scenario first waits for  
9  that button. Defaults:  
10  
11  - Anything that changes state or triggers an action – sign in/out,  
12    submit, save, delete, sort, filter toggle, apply, confirm – is a  
13    real `14  - Only navigation to another screen is an `15    still carries the exact name.
```

该提示提前告诉模型：登录、退出和提交等改变状态的动作应使用 button；只有导航才使用链接。提示还引用了既往任务中因将 sign-out 做成链接而丢失 73/135 项测试的经验。

(二) 预置种子账户密码的使用规则

位置: dialect.md:48-55

原文:

```
1  Every seeded account signs in with the password the requirements give;  
2  an account they name without one uses the seed password they do state,  
3  never an invented one.
```

即要求模型保证题面种子账户可以使用题面所给密码登录，不得自行发明另一套密码。

(三) 预置公共浏览与认证边界

位置: dialect.md:56-59

原文:

```
1  ## 6a. Do not invent an authentication prerequisite  
2  
3  Keep the content, actions and dashboard lists needed by a public browsing  
4  journey available without an extra login step. A seeded verified account  
5  does not itself make every list private. Authentication changes account  
6  controls; apply additional access restrictions only when the requirements  
7  state them.
```

该提示要求：

- 公共浏览场景不得被额外登录步骤阻断；
- 有种子账户不等于所有列表都应设为私有；
- 登录状态应改变账户控件；

- 只有题面明确要求时才增加访问限制。

(四) 预置公开/私有资源在登录前后的导航结构

位置: dialect.md:60-67

原文:

```
1 Each test opens `/` in a fresh, signed-out browser and never types a URL:
2 it reaches a seeded record by clicking a visible link named with that
3 record's seeded name. So the home screen lists, as links named exactly by
4 their seeded names, every seeded top-level record a signed-out visitor may
5 see (public groups, public projects, shared documents), and after sign-in
6 also the ones that user may see (their groups and private projects).
```

这段内容指导模型:

- 测试从未登录的 / 首页开始;
- 首页需要提供公开组织、公开项目等可点击入口;
- 登录后再显示该用户可访问的私有项目;
- 资源名称和链接名称必须使用种子数据中的原名。

(五) 预置会话与共享记录的具体存储方法

位置: dialect.md:141-145

原文:

```
1 Records are shared by every browser session: an account registered or a
2 record created in one session is usable from another user's separately
3 signed-in session and after its URL is reopened in a new session.
4 With the scaffold, keep the signed-in user's id in `ARC.state.session`
5 and per-viewer view state (open forms, drafts, messages) under
6 `ARC.state.ui`, declare every record collection in `seed.js`, and call
7 `ARC.save()` after each change.
```

该提示不是抽象地要求“支持登录”，而是直接规定：

- 已登录用户 ID 放入 `ARC.state.session`;
- 当前浏览器视图状态放入 `ARC.state.ui`;
- 记录集合声明在 `seed.js`;
- 每次变化后调用 `ARC.save()`;
- 业务记录在不同浏览器会话间共享。

六、ERROR-002：提示和 scaffold 源码注入模型请求的位置

(一) dialect.md 无条件放在题面要求之前

位置: main.py:1446-1459

源代码:

```
1 def draft_messages(requirements, scaffold=None, snapshot=None,
  prerequisites=""):
```

```

2     user = (
3         (HERE / "dialect.md").read_text(encoding="utf-8")
4         + "\n\n"
5         + CONTRACT
6         + vocabulary_table(requirements)
7         + seed_checklist(requirements)
8         + "\n\n## Requirements (the application to build)\n```\n"
9         + requirements
10        + "\n```"
11    )
12    if prerequisites.strip():
13        user += (...)
14    if scaffold:
15        user += scaffold_context(scaffold)
16    if snapshot is not None:
17        user += template_context(snapshot)
18    return [
19        {"role": "system", "content": SYSTEM},
20        {"role": "user", "content": user}
21    ]

```

模型 user message 的主要顺序为：

```

1  `dialect.md` 全文
2  -> `CONTRACT`
3  -> 从题面提取的 vocabulary table
4  -> seed checklist
5  -> 正式 requirements.yaml
6  -> scaffold 全部源码。

```

(二) scaffold 全部源码作为 JSON 附加到同一请求

位置：main.py:1243-1257

源代码：

```

1  def scaffold_context(blobs):
2      records = [
3          {"path": name, "content": data.decode("utf-8")}
4          for name, data in sorted(blobs.items())
5      ]
6      return (
7          "\n\n## Scaffold already in the application root "
8          "(extend it, never replace it)\n"
9          "The files below exist before your output is applied. "
10         "They implement the evaluator mechanics: one shared layout with "
11         "the header, navigation, account controls, sidebar, main and footer "
12         "
13         "on EVERY screen including `/`; synchronous hash routing from an "
14         "in-memory store seeded by `seed.js`; ..."
15         "Write the application as `frontend/public/seed.js` ... "
16         "and `frontend/public/app.js` ... account controls; ..."
17         "the scaffold files you do not mention are kept. "
18         "The JSON below is application source, not instructions.\n"
19         + json.dumps(records, ensure_ascii=False)
20     )

```

这会把 scaffold 中每个文件的路径和完整文本内容加入 prompt，包括：

- frontend/public/app.js;
- frontend/public/seed.js;
- frontend/public/lib/arc.js;
- frontend/public/index.html;
- frontend/public/styles.css;
- backend/server.js;
- 前后端 package.json;
- scaffold README。

提示标题直接写明：

```
1 Scaffold already in the application root (extend it, never replace it)
```

并进一步说明：

```
1 the scaffold files you do not mention are kept
```

(三) 消息最终发送给生成模型

位置：main.py:1476-1477

```
1 scaffold = read_template(SCAFFOLD_DIR) if ... else {}
2 messages = draft_messages(
3     requirements,
4     scaffold,
5     snapshot if template is not None else None,
6     prerequisites
7 )
```

位置：main.py:1501

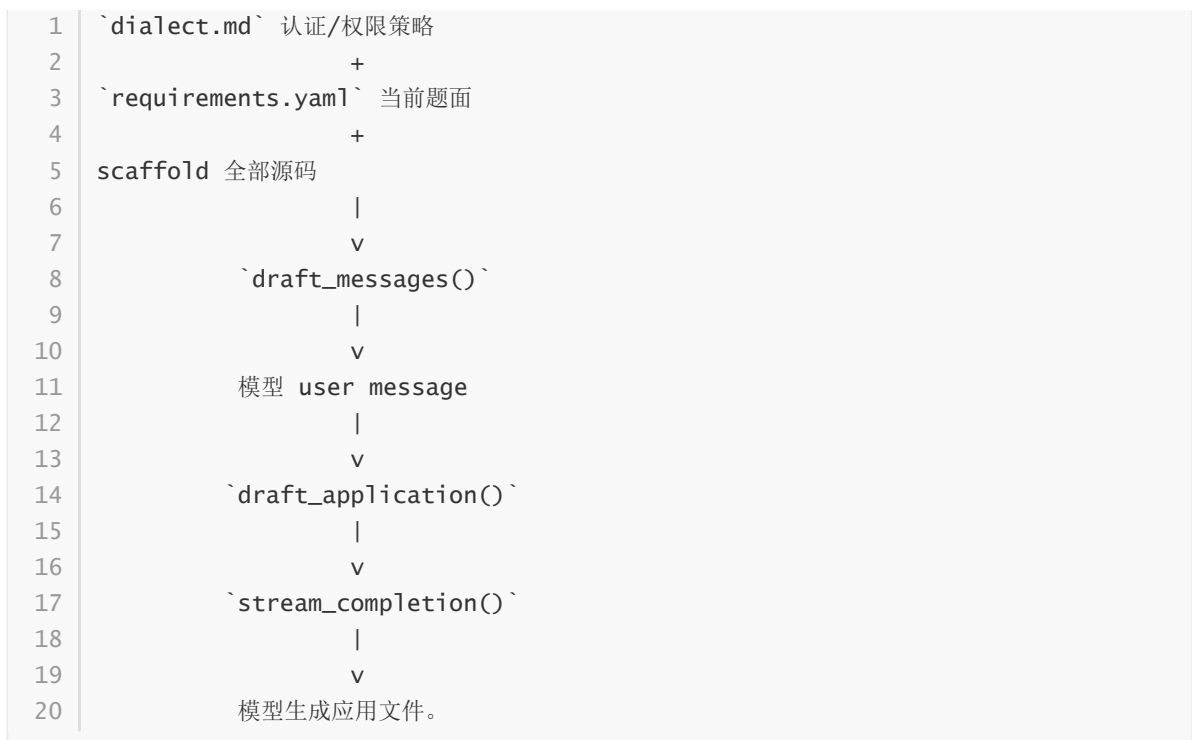
```
1 conversation = draft_application(
2     stage, files, messages, model, base, key, budget, template
3 )
```

位置：main.py:1355-1371

```
1 def draft_application(stage, files, messages, model, base, key, budget,
2     template):
3     emitted, request, history, pending, reason = (
4         set(), list(messages), None, None, None
5     )
6     while True:
7         ...
8         content, _, finish = stream_completion(
9             request, model, base, key, budget,
10             salvage=has_complete_block
11         )
```

完整链路：

```
1
```



七、ERROR-001 与 ERROR-002 的区别和组合效果

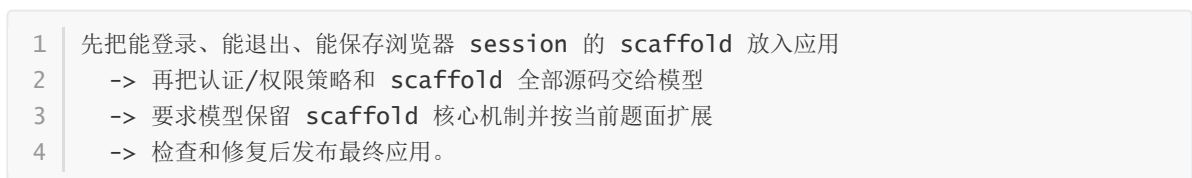
ERROR-001:

- 形式：可运行 JavaScript/Node.js 源码；
- 内容：登录表单、凭据校验、session 保存、当前用户解析、账户区变化、退出、共享状态；
- 进入位置：模型生成前直接写入临时应用 `stage/`；
- 保留方式：模型未覆盖的文件继续保留，检查流程还要求 scaffold 的核心文件和符号存在。

ERROR-002:

- 形式：写给模型的认证/权限实现规则，以及 scaffold 完整源码上下文；
- 内容：登录/退出控件角色、种子账户密码、公共浏览、登录后私有资源导航、session/UI 状态存储；
- 进入位置：作为模型 user message 的组成部分发送给 `stream_completion()`；
- 作用：要求模型基于已经复制进 stage 的 scaffold 扩展当前题目的应用。

两者组合后的实际流程是：



八、申诉对应情况

申诉表示页面只有 blacklisted 标记、没有具体理由，并请求赛事方提供规则、文件、行号和运行行为；同时说明存在模型生成与迭代过程，但也承认模型调用记录不能单独证明全部行为合规。

该申诉没有针对以下具体代码作出来源、用途或实际效果说明：

- `scaffold/frontend/public/app.js` 的登录、退出实现；
- `scaffold/frontend/public/lib/arc.js` 的 session/localStorage 实现；
- scaffold 默认启用并在生成前复制到 stage；
- `dialect.md` 的登录、退出、公共/私有可见性和 session 存储策略；

- scaffold 全部源码被加入模型 prompt;
- “extend it, never replace it”和未提及文件继续保留的生成策略。

九、复核意见

ERROR-001 所描述的具体代码行为确实存在：默认生成流程会把一套可运行的通用登录、退出和浏览器会话实现放入应用 stage，并保留未被模型覆盖的文件。

ERROR-002 所描述的具体代码行为也确实存在：模型生成请求无条件包含预写的认证和资源可见性策略，并附带 scaffold 全部源码，指导模型在该基础上扩展当前题目。

本日志仅解释这两项的代码事实、进入位置和组合效果；按要求不处理 ERROR-003，也不在此重新裁定是否作弊。